As with the acceptance specs, you'll be using Rack::Test to route HTTP requests to the API class. The other big change is packaging up the status information in a simple RecordResult class. Eventually, we'll move this definition into the application code. But that can wait until we've defined Ledger.

**Use Value Objects at Layer Boundaries**



The seam between layers is where integration bugs hide. Using a simple value object like a RecordResult or Struct between layers makes it easier to isolate code and trust your tests. See Gary Bernhardt's excellent "Boundaries" talk for more details.[5]

Now, you're ready to fill in the body of the first example. Inside the first context, find the empty 'returns the expense id' spec that you sketched out earlier. Change it to the following code:

05-unit-specs/03/expense_tracker/spec/unit/app/api_spec.rb

```
it 'returns the expense id' do
  expense = { 'some' => 'data' }

  allow(ledger).to receive(:record)
    .with(expense)
    .and_return(RecordResult.new(true, 417, nil))

  post '/expenses', JSON.generate(expense)

  parsed = JSON.parse(last_response.body)
  expect(parsed).to include('expense_id' => 417)
end
```

On the highlighted lines, we're calling the allow method from rspec-mocks. This method configures the test double's behavior: when the caller (the API class) invokes record, the double will return a new RecordResult instance indicating a successful posting.

Another thing to note: The expense hash we're passing doesn't look anything like valid data. In the live app, the incoming data will look more like { 'payee' => ..., 'amount' => ..., 'date' => ... }. This is okay; the whole point of the Ledger test double is that it will return a canned success or failure response, no matter the input.

Having data that looks obviously fake can actually be a big help. You'll never mistake it for the real thing in your test output and waste time wondering, "How did *this* expense result in *that* report?"

Run your specs; they should fail, because the API behavior is not implemented yet:

5. https://www.destroyallsoftware.com/talks/boundaries